

RNN Journey

Concept Notes — Vanilla RNN

Deep Learning Roadmap: ANN → CNN → RNN → NLP → Transformer → LLM → Fine-tuning

Student notes • diagrams • simple-word explanations

Table of Contents

1.	What RNN Actually Does	3
2.	The Memory Box Idea (Diagram)	3
3.	What Is Fading?	4
4.	Real Example — Fading in Action	5
5.	LSTM — The Smarter Memory (Diagram)	6
6.	LSTM Concept — Simple Words	7
7.	LSTM Math — Worked Example by Hand	8

1 | What RNN Actually Does

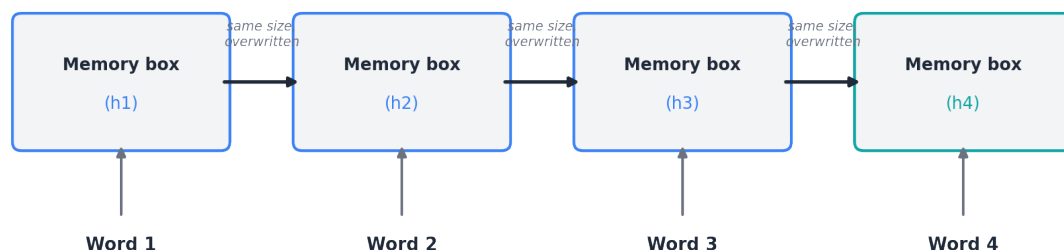
- **RNN reads things one at a time** — like reading a sentence word by word, not all at once.
- It keeps a **small memory box** — after every word, it updates this memory box with a short summary of everything so far.
- The memory box is **always the SAME SIZE** — doesn't matter if you read 5 words or 500 words, the box size never changes.
- **New word + old memory = new memory** — every time a new word comes in, RNN mixes it with the old memory to make a new updated memory.
- **Old memory keeps getting overwritten** — because the box is small and fixed size, older information slowly fades away as new words keep coming in.
- **Recent words are remembered well** — because they were added most recently, they haven't faded yet.
- **Old words are remembered poorly** — the further back a word was, the more its info has faded/mixed away.

KEY POINT: This fading is NOT based on importance — RNN doesn't know "this word is important, protect it." It just fades based on how long ago the word appeared.

RNN'S BIGGEST WEAKNESS — called the "**vanishing gradient problem**" — it forgets old stuff even if that old stuff was actually really important. This weakness is exactly why **LSTM** was invented — LSTM adds a smarter memory system that CAN protect important old information.

2 | The Memory Box Idea

RNN memory box: same fixed size at every step, constantly overwritten



The same memory box gets passed forward and overwritten at every timestep — it never grows in size.

3 | What Is Fading?

Fading = the NUMBERS getting smaller, literally. The part of the memory that came from an OLD word keeps shrinking closer to zero every time a new word gets mixed in.

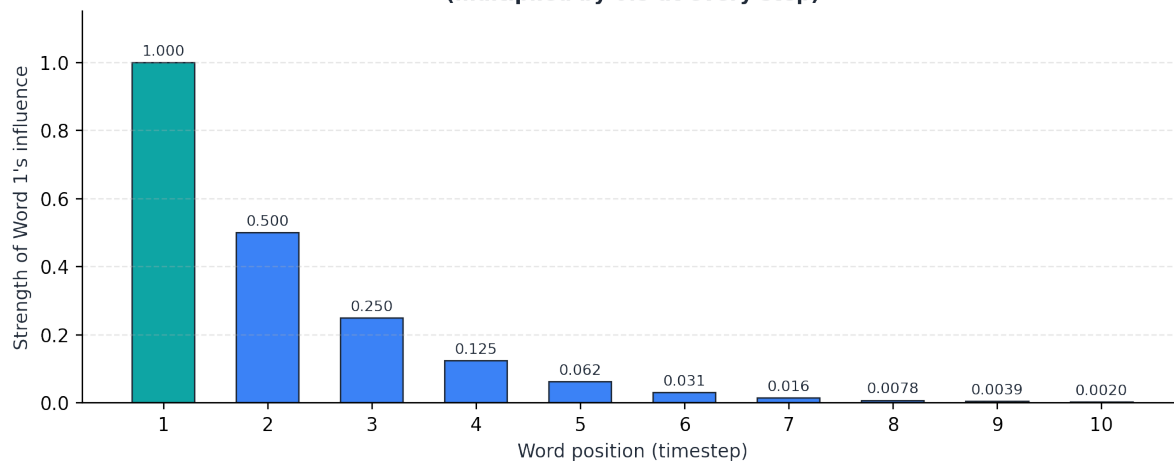
Simple example — one number, not four

Say the hidden state is just ONE number. Every time a new word is mixed in, the OLD memory gets multiplied by 0.5 (this comes from the weights + tanh squashing — real numbers vary, but 0.5 shows the pattern clearly):

Word arrives	Old memory $\times 0.5$	New memory
Word 1	—	1.0 (full strength)
Word 2	1.0×0.5	0.5
Word 3	0.5×0.5	0.25
Word 4	0.25×0.5	0.125
Word 5	0.125×0.5	0.0625
Word 10	...continues...	≈ 0.001 (practically zero)

KEY POINT: Word 1's original strength (1.0) kept getting cut in HALF every single step. By word 10, word 1's contribution is basically invisible — **this shrinking process is called "fading."** It's not that the word is deleted — its numerical influence just becomes too tiny to matter.

**Fading: Word 1's influence shrinks as more words arrive
(multiplied by 0.5 at every step)**

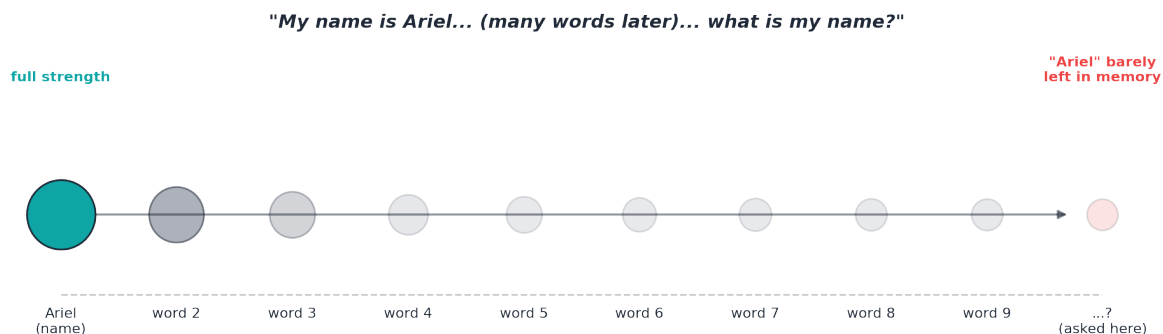


Word 1's influence shrinks exponentially — it never truly hits zero, but becomes practically meaningless.

4 | Real Example — Fading in Action

Sentence: “My name is Ariel. [...lots of other words/names about yourself...]”

- **Word 1: “Ariel”** arrives → gets mixed into memory at FULL strength (like the 1.0 example).
- Then more words come: “I am from...”, “I like...”, “I work as...”, other names, other details...
- Each NEW word mixes in, and the OLD memory (which contains “Ariel”) gets multiplied by a small number again and again.
- By the time 50–100 more words are said, “Ariel”'s influence in the memory has shrunk close to ZERO — **not because “Ariel” isn't important, but simply because it's now far away (in position) from the current word.**



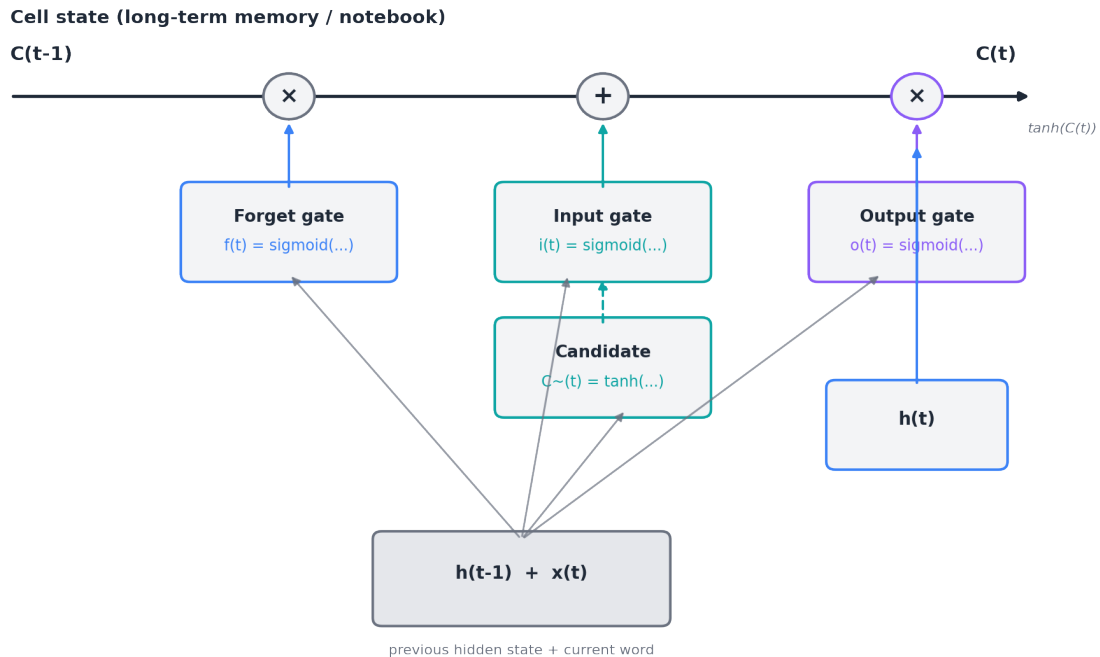
Circle size/color = how much of “Ariel” is still left in memory at that word position.

REAL-WORLD CONSEQUENCE: If someone then asks the RNN “what's my name?” after 100 words, vanilla RNN would likely struggle to answer correctly — because “Ariel”'s signal has faded too much by then, even though it's literally the answer.

WHY LSTM FIXES THIS: This is the exact real-world problem LSTM was built to fix — LSTM's gates would recognize “Ariel” as worth protecting (since it learned during training that names are often asked about later) and keep it strong in the cell state / notebook, instead of letting it mechanically fade.

5 | LSTM — The Smarter Memory

LSTM (Long Short-Term Memory) is RNN's upgrade — it adds a protected long-term “notebook” (cell state) alongside the short-term summary (hidden state), controlled by three gates.



C = cell state (long-term) h = hidden state (short-term) f, i, o = gates $\tilde{C}$ = candidate

Cell state (top) runs mostly straight through — only touched by $\times$ (forget) and $+$ (input), never squashed every step like RNN's hidden state.

6 | LSTM Concept — Simple Words

- **LSTM is RNN's smarter cousin** — same idea of reading one word at a time and keeping memory, but with a much better memory system.
- **It has TWO memories instead of one** — a **notebook (cell state, C)** for long-term important stuff, and a **short-term summary (hidden state, h)** for what's happening right now.
- **The notebook is protected** — unlike RNN's memory box which gets overwritten completely every step, the notebook only changes when a gate EXPLICITLY decides to erase or write something.
- **Three gates control everything** — three small guards standing at the notebook:
 - **Forget gate** — “Should I erase old stuff from the notebook?”
 - **Input gate** — “Should I write this new stuff into the notebook?”
 - **Output gate** — “What part of the notebook should I use right now?”
- **Gates are LEARNED, not fixed** — during training, the network learns WHEN to open or close each gate — e.g. “names are usually important, keep them” or “filler words like 'the', 'is' aren't important, forget them.”

KEY DIFFERENCE FROM RNN: RNN fades EVERYTHING equally based on distance alone. LSTM can choose to protect important info regardless of how far back it was said.

THE “ARIEL” EXAMPLE, REVISITED: LSTM's input gate would recognize “Ariel” as worth writing into the notebook. Even 100 words later, the forget gate keeps that entry mostly untouched (because it learned names matter) — so when asked “what's my name?”, the info is still there.

Why the name “Long Short-Term Memory”? Because it combines BOTH: short-term memory (hidden state, like RNN) AND a mechanism for long-term memory (cell state protected by gates) — “long” + “short-term” memory, together in one name.

7 | LSTM Math — Worked Example by Hand

Every gate follows the same pattern: **gate** = **sigmoid**($W \cdot h(t-1) + U \cdot x(t) + b$) — W , U are learned weights, b is bias. Candidate values use tanh instead of sigmoid.

Setup (treating everything as a single number to keep it simple): $h(t-1) = 0.5$, $C(t-1) = 0.8$, $x(t) = 1.0$

Step	Formula	Calculation	Result
1. Forget gate	$f(t) = \text{sigmoid}(W_f \cdot h + U_f \cdot x + b_f)$	$\text{sigmoid}(0.5 \cdot 0.5 + 0.3 \cdot 1.0 + 0.1)$	$f(t) = 0.657$
2. Input gate	$i(t) = \text{sigmoid}(W_i \cdot h + U_i \cdot x + b_i)$	$\text{sigmoid}(0.4 \cdot 0.5 + 0.6 \cdot 1.0 - 0.1)$	$i(t) = 0.668$
3. Candidate	$\tilde{C}(t) = \tanh(W_c \cdot h + U_c \cdot x + b_c)$	$\tanh(0.3 \cdot 0.5 + 0.5 \cdot 1.0 + 0.05)$	$\tilde{C}(t) = 0.604$
4. New cell state	$C(t) = f(t) \cdot C(t-1) + i(t) \cdot \tilde{C}(t)$	$(0.657 \cdot 0.8) + (0.668 \cdot 0.604)$	$C(t) = 0.929$
5. Output gate	$o(t) = \text{sigmoid}(W_o \cdot h + U_o \cdot x + b_o)$	$\text{sigmoid}(0.6 \cdot 0.5 + 0.2 \cdot 1.0 + 0)$	$o(t) = 0.622$
6. Hidden state	$h(t) = o(t) \cdot \tanh(C(t))$	$0.622 \cdot \tanh(0.929)$	$h(t) = 0.455$

KEY POINT: In Step 4, the old notebook ($C(t-1)$) is only **partially forgotten** ($\times f(t)$) and new info is only **partially added** ($\times i(t)$) — nothing gets squashed through tanh here. That's exactly why old information survives much longer in LSTM than in vanilla RNN, where the ENTIRE hidden state gets squashed through tanh at every single step.

Result: $h(t) = 0.455$ (new short-term memory) and $C(t) = 0.929$ (updated notebook) both get carried forward into the next timestep.